

Pydantic

It is widely used for data visualization and a settings management library for Python, designed to enforce type hints at runtime.

It leverage Rust for high-performance validation and serialization, making it ideal for data parsing, API development (FastAPI), and AI application, offering strict/lax modes and seamless IDE integration.

Key Features & Capabilities

- **Data Validation:** Ensures data adheres to specified types and constraints (e.g., `int`, `str`, `list`) during object instantiation.
- **Fast Performance:** Core validation logic is written in Rust.
- **JSON Schema:** Models can easily generate JSON Schema, aiding integration with other tools.
- **Type Hinting:** Uses standard Python 3.9+ type annotations to define data structures.
- **Pydantic AI:** A framework for building production-grade AI agents, providing type-safe LLM applications.
- **Pydantic Logfire:** A monitoring tool for tracing LLM calls, debugging, and tracking token costs

BaseModel

BaseModel is the fundamental building block used to define data structures with automatic validation and type enforcement.

By creating a class that inherits from BaseModel, you define "fields" using python type hints, and Pydantic ensures any data used to create an instance of that matches those types. In recursive term, you create data models by inheriting from it.

```
from pydantic import BaseModel

class User(BaseModel):
    id: int
    name: str
```

-->

```
user = User(id="123", name="Alice")

print(user)
```

Then the output would be

```
id=123 name='Alice'
```

And if wrong data is passed:

```
User(id="abc", name="Alice")
```

Then it would raise an error

```
ValidationError:  
id  
  Input should be a valid integer
```

And Pydantic is intelligently convert values

```
User(id="123", name="Alice")
```

The id = 123

but

```
User(id="hello", name="Alice")
```

--> Fail

Convert Model to dictionary

```
user = User(id=1, name="Alice")  
  
print(user.model_dump())
```

Output

```
{'id': 1, 'name': 'Alice'}
```

Convert to JSON

```
user.model_dump_json()
```

Output

```
{"id":1,"name":"Alice"}
```

It is very useful for the APIs

Nested Models

Pydantic automatically converts nested dictionaries into objects.

```
from pydantic import BaseModel

class Address(BaseModel):
    city: str
    country: str

class User(BaseModel):
    name: str
    address: Address
```

Usage:

```
data = {
    "name": "John",
    "address": {
        "city": "London",
        "country": "UK"
    }
}

user = User(**data)

print(user.address.city)
```

Output

London

Note: This is what **does** ``

```
api_response = {
    "name": "Alice",
    "age": 30,
    "email": "alice@example.com"
}
```

```
user = User(api_response)
```

#Equivalent to writing:

```
user = User(name="Alice", age=30, email="alice@example.com")
```

This takes a dictionary and spreads its key-value pairs as keyword arguments into a function or class constructor.

```
from pydantic import BaseModel, Field

class Response(BaseModel):
    answer: str = Field()
```

```
description="A natural language response to the user's question."  
)
```

Field = used to provide metadata, validation rules, and configuration for a model field.